UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ingeniería en Tecnologías de la Información e Innovación Digital

ACTIVIDAD 5
Búsqueda heurística en una red vial

Datos de entrega	Información
Asignatura	Fundamentos de Inteligencia Artificial
Integrantes	Bryan Salvador Ramos López · Alexis Kevin Pérez García · Jorge Lidio Barragán Villanueva · Emmanuel Guzmán García
Grupo	172
Docente	Mtro. Javier García Lara
Fecha indicada en el enunciado	25 de octubre de 2026

Trabajo en equipo versionado en GitHub. Se incluyen planteamiento, procedimiento, evidencia, conclusiones y referencias.

Planteamiento y datos

Caso académico: hallar una ruta peatonal entre dos nodos de la red vial de Atlautla, Estado de México. Las calles y coordenadas provienen de una instantánea de OpenStreetMap; la necesidad de viaje y sus extremos son un escenario hipotético. No es una indicación de navegación real ni garantiza accesibilidad o seguridad.

El grafo no dirigido conexo contiene 851 intersecciones/nodos y 916 segmentos. Cada arista pesa su longitud geodésica aproximada en metros. Inicio: nodo 5670007917; meta: nodo 5669889529. Se eligió el inicio próximo a una coordenada de referencia del campus y la meta como el nodo más distante en línea recta dentro del recorte.

Formulación y método

Estado: nodo vial actual. Acción: avanzar a un nodo adyacente. Costo: longitud del segmento. Objetivo: minimizar la suma de longitudes hasta la meta. Dijkstra prioriza g(n), el costo recorrido; A* prioriza f(n)=g(n)+h(n). Se usó $h(n)=0.999 \cdot d_{Haversine}(n,meta)$. El factor 0.999 compensa el redondeo de pesos a milímetros. En cada arista, la distancia geodésica directa no supera la longitud del tramo; con ese factor, h es consistente y admisible en la instantánea almacenada.

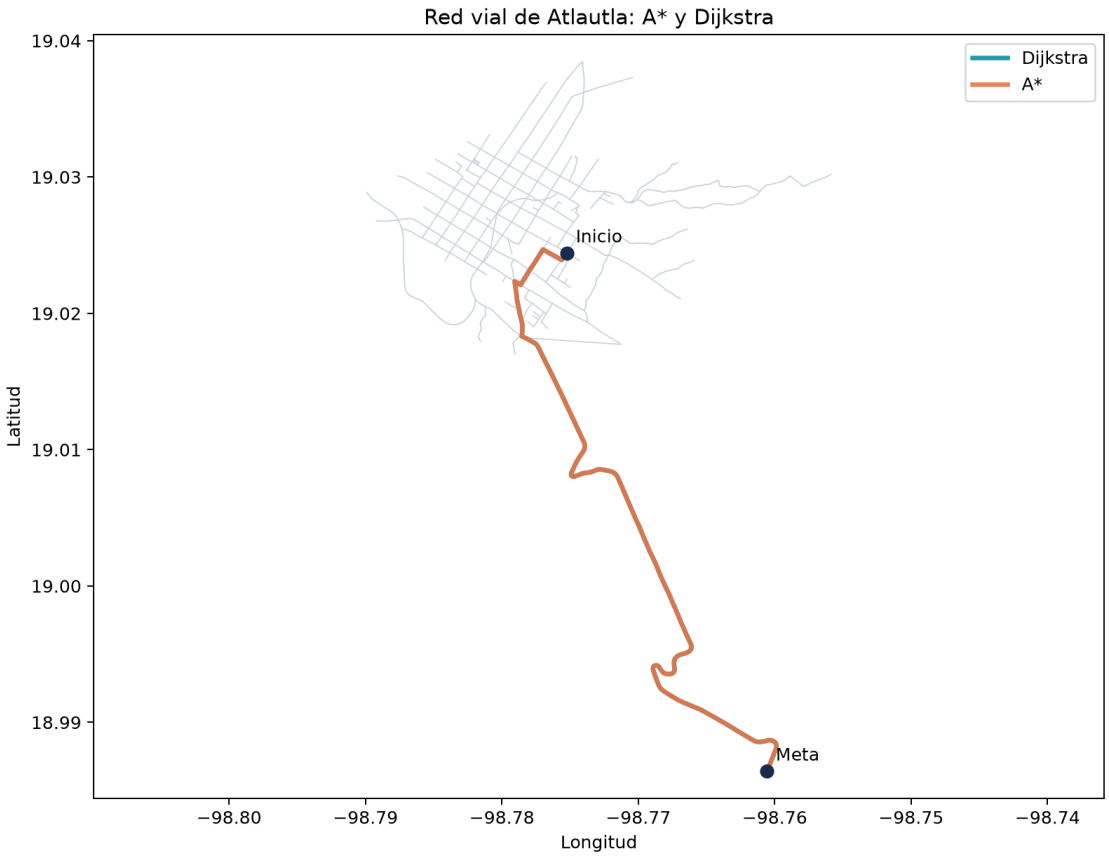
Procedimiento reproducible

1) Descargar la caja geográfica mediante **Act5/obtener_red_osm.py** (la instantánea JSON ya se incluye). 2) Filtrar vías caminables y conservar la componente conexa mayor. 3) Ejecutar ambos algoritmos con los mismos extremos. 4) Registrar ruta, distancia, estados expandidos, frontera y mediana de 50 ejecuciones en milisegundos. 5) Comparar sobre el mismo mapa. Ejecutar: `python Act5/act5_rutas.py`.

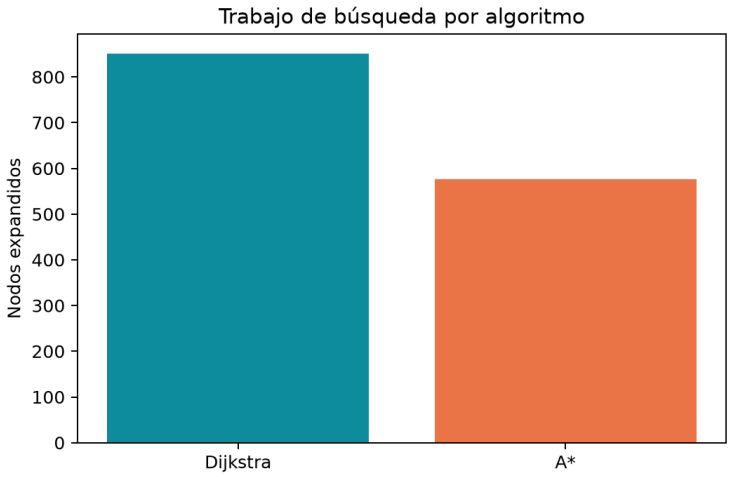
Métrica	Dijkstra	A*
Longitud (m)	6207.666	6207.666
Nodos expandidos	851	576
Frente máximo	32	23
Mediana de 50 tiempos (ms)	2.6089	3.7630

Ambos algoritmos obtuvieron exactamente el mismo costo en este caso. A* expandió menos estados, pero aquí su cálculo de la heurística elevó el tiempo mediano. Los tiempos dependen del equipo y no son una promesa de ventaja universal.

Mapa y tablero



El tablero interactivo Plotly se encuentra en **Act5/dashboard.html**. La interfaz principal del repositorio permite cambiar los extremos y repetir la comparación en el navegador. El código Python está comentado y se anexa.



Complejidad y límites

Con listas de adyacencia y montículo, la cota usual de ambos métodos es O((V+E) log V) en tiempo y O(V+E) en memoria. A* puede expandir menos estados si la heurística es informativa, pero su costo de evaluación importa. La red omitió restricciones de sentido y accesibilidad; se modeló como no dirigida.

Conclusión

La heurística admisible mantuvo la ruta óptima y redujo expansiones en esta instancia. Para una aplicación de transporte real harían falta velocidades, sentidos de circulación, cierres y validación de campo.

Referencias (APA 7.ª ed.)

OpenStreetMap contributors. (2026). OpenStreetMap [datos geográficos]. https://www.openstreetmap.org/copyright

NetworkX developers. (s. f.). Shortest paths. https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html

Russell, S., & Norvig, P. (2020). Artificial intelligence: A modern approach (4th ed.). Pearson.

Uso de inteligencia artificial y verificación

Se utilizó ChatGPT/Codex para apoyar la organización del reporte, la redacción, el desarrollo de código y gráficos. Se ejecutaron los programas y pruebas locales, y se cotejaron las afirmaciones documentales con las fuentes citadas. Quienes presentan el trabajo deben revisarlo, comprenderlo y poder explicarlo.

Anexo: código Python completo

Archivo ejecutable **Act5/act5_rutas.py**. La numeración facilita la revisión; el archivo del repositorio es la versión directamente ejecutable.

```
001  """A* y Dijkstra sobre una instantánea real de calles de Atlautla.
002
003  La red es una aproximación caminable no dirigida; los extremos son nodos viales
004  reales, pero el motivo del viaje es un escenario académico simulado.
005  """
006  from __future__ import annotations
007
008  import heapq
009  import json
010  import math
011  import statistics
012  import time
013  from pathlib import Path
014
015  import matplotlib
016  matplotlib.use("Agg")
017  import matplotlib.pyplot as plt
018  import networkx as nx
019  import plotly.graph_objects as go
020  from plotly.subplots import make_subplots
021
022  from obtener_red_osm import haversine
023
024  BASE = Path(__file__).resolve().parent
025  RED = BASE / "datos" / "red_atlautla.json"
026  OUT = BASE / "resultados"
027  CAMPUS_APROX = (19.024609, -98.77549)
028
029
030  def cargar_red() -> tuple[nx.Graph, dict]:
031      datos = json.loads(RED.read_text(encoding="utf-8"))
032      g = nx.Graph()
033      for identificador, (lat, lon) in datos["nodos"].items():
034          g.add_node(identificador, lat=lat, lon=lon)
035      for a, b, metros in datos["aristas"]:
036          g.add_edge(a, b, weight=metros)
037      return g, datos
038
039
040  def punto(g: nx.Graph, nodo: str) -> tuple[float, float]:
041      return g.nodes[nodo]["lat"], g.nodes[nodo]["lon"]
042
043
044  def mas_cercano(g: nx.Graph, latlon: tuple[float,float]) -> str:
045      return min(g.nodes, key=lambda n: (haversine(punto(g, n), latlon), n))
046
047
048  def extremos(g: nx.Graph) -> tuple[str, str]:
049      inicio = mas_cercano(g, CAMPUS_APROX)
050      meta = max(g.nodes, key=lambda n: (haversine(punto(g, inicio), punto(g, n)), n))
051      return inicio, meta
052
053
054  def buscar(g: nx.Graph, inicio: str, meta: str, algoritmo: str) -> dict:
055      if algoritmo not in {"Dijkstra", "A*"}:
056          raise ValueError("Algoritmo no reconocido")
057      objetivo = punto(g, meta)
058      # Factor conservador por el redondeo a milímetros de los pesos del JSON.
059      def h(n: str) -> float:
060          return 0.999 * haversine(punto(g, n), objetivo) if algoritmo == "A*" else 0.0
061      cola = [(h(inicio), h(inicio), inicio)]
062      dist = {inicio: 0.0}
063      padres = {inicio: None}
064      cerrados = set()
065      max_cola = 1
066      while cola:
067          _, _, actual = heapq.heappop(cola)
068          if actual in cerrados:
069              continue
070          cerrados.add(actual)
071          if actual == meta:
072              ruta = []
073              while actual is not None:
074                  ruta.append(actual)
075                  actual = padres[actual]
076              ruta.reverse()
077              return {"ruta": ruta, "distancia_m": round(dist[meta], 3),
078                      "nodos_expandidos": len(cerrados), "max_frontera": max_cola}
079          for vecino in sorted(g.neighbors(actual)):
080              if vecino in cerrados:
081                  continue
```

```
            nuevo = dist[actual] + g[actual][vecino]["weight"]
            if nuevo < dist.get(vecino, math.inf) - 1e-9:
                dist[vecino] = nuevo
                padres[vecino] = actual
                estimacion = h(vecino)
                heapq.heappush(cola, (nuevo + estimacion, estimacion, vecino))
        max_cola = max(max_cola, len(cola))
    raise ValueError("No existe una ruta dentro del grafo")


def medir(g: nx.Graph, inicio: str, meta: str, algoritmo: str, repeticiones: int = 50) -> tuple[dict, float]:
    muestras = []
    resultado = None
    for _ in range(repeticiones):
        t0 = time.perf_counter_ns()
        resultado = buscar(g, inicio, meta, algoritmo)
        muestras.append((time.perf_counter_ns() - t0) / 1e6)
    return resultado, round(statistics.median(muestras), 4)


def dibujar(g: nx.Graph, rutas: dict[str,dict]) -> None:
    OUT.mkdir(exist_ok=True)
    fig, ax = plt.subplots(figsize=(9, 7))
    for a, b in g.edges:
        pa, pb = punto(g, a), punto(g, b)
        ax.plot([pa[1], pb[1]], [pa[0], pb[0]], color="#cbd5df", lw=.7)
    colores = {"Dijkstra": "#0d8c9d", "A*": "#eb7447"}
    for nombre, res in rutas.items():
        coords = [punto(g, n) for n in res["ruta"]]
        ax.plot([v[1] for v in coords], [v[0] for v in coords],
                color=colores[nombre], lw=2.6, label=nombre, alpha=.9)
    for nodo, etiqueta in ((rutas["A*"]["ruta"][0], "Inicio"),
                           (rutas["A*"]["ruta"][-1], "Meta")):
        lat, lon = punto(g, nodo)
        ax.scatter(lon, lat, s=50, color="#172b4d", zorder=5)
        ax.annotate(etiqueta, (lon, lat), xytext=(5, 6), textcoords="offset points")
    ax.set(xlabel="Longitud", ylabel="Latitud", title="Red vial de Atlautla: A* y Dijkstra")
    ax.legend(); ax.set_aspect("equal", adjustable="datalim")
    fig.tight_layout(); fig.savefig(OUT / "act5_rutas.png", dpi=180); plt.close(fig)

    fig, ax = plt.subplots(figsize=(6, 4))
    ax.bar(["Dijkstra", "A*"], [rutas[k]["nodos_expandidos"] for k in ("Dijkstra", "A*")],
           color=[colores["Dijkstra"], colores["A*"]])
    ax.set(ylabel="Nodos expandidos", title="Trabajo de búsqueda por algoritmo")
    fig.tight_layout(); fig.savefig(OUT / "act5_comparacion.png", dpi=180); plt.close(fig)


def dashboard(g: nx.Graph, rutas: dict[str,dict]) -> None:
    fig = make_subplots(rows=1, cols=2, column_widths=[.73, .27],
                        specs=[[{"type":"scatter"},{"type":"bar"}]],
                        subplot_titles=("Mapa de segmentos y rutas", "Nodos expandidos"))
    xs, ys = [], []
    for a, b in g.edges:
        pa, pb = punto(g,a), punto(g,b)
        xs.extend([pa[1],pb[1],None]); ys.extend([pa[0],pb[0],None])
    fig.add_trace(go.Scatter(x=xs,y=ys,mode="lines",line=dict(color="#c7d2df",width=1),
                             name="Calles OSM",hoverinfo="skip"), row=1,col=1)
    for nombre,color in (("Dijkstra","#0d8c9d"),("A*","#eb7447")):
        coords=[punto(g,n) for n in rutas[nombre]["ruta"]]
        fig.add_trace(go.Scatter(x=[v[1] for v in coords],y=[v[0] for v in coords],
                                 mode="lines+markers",marker=dict(size=4),
                                 line=dict(color=color,width=4),name=nombre,
                                 hovertemplate="Lat %{y:.5f}, lon %{x:.5f}<extra>"+nombre+"</extra>"),row=1,col=1)
    fig.add_trace(go.Bar(x=["Dijkstra","A*"],
                         y=[rutas[k]["nodos_expandidos"] for k in ("Dijkstra","A*")],
                         marker_color=["#0d8c9d","#eb7447"],showlegend=False),row=1,col=2)
    fig.update_layout(title="Act5 · A* y Dijkstra en Atlautla",height=630,
                      template="plotly_white",font=dict(family="Arial"),
                      legend=dict(orientation="h",y=-.09))
    fig.update_xaxes(title_text="Longitud",row=1,col=1)
    fig.update_yaxes(title_text="Latitud",row=1,col=1,scaleanchor="x",scaleratio=1)
    fig.update_yaxes(title_text="Estados",row=1,col=2)
    fig.write_html(BASE / "dashboard.html",include_plotlyjs=True,full_html=True)


def ejecutar() -> dict:
    g, fuente = cargar_red()
    inicio, meta = extremos(g)
    rutas = {}
    for algoritmo in ("Dijkstra", "A*"):
        rutas[algoritmo], tiempo = medir(g, inicio, meta, algoritmo)
        rutas[algoritmo]["tiempo_mediana_ms_50"] = tiempo
        rutas[algoritmo]["coordenadas_ruta"] = [punto(g,n) for n in rutas[algoritmo]["ruta"]]
    dibujar(g, rutas)
    dashboard(g, rutas)
```

```
    datos = {"fuente": fuente["url"], "fecha_descarga_utc": fuente["fecha_descarga_utc"],
             "modelo": fuente["nota"], "nodos": g.number_of_nodes(),
             "aristas": g.number_of_edges(), "inicio": {"id":inicio,"latlon":punto(g,inicio)},
             "meta": {"id":meta,"latlon":punto(g,meta)}, "resultados": rutas,
             "heuristica": "0.999 × Haversine en metros; factor conservador frente al redondeo de aristas."}
    (OUT / "act5_resultados.json").write_text(json.dumps(datos,ensure_ascii=False,indent=2),encoding="utf-8")
    print(json.dumps({k:{"distancia_m":v["distancia_m"],"nodos_expandidos":v["nodos_expandidos"],
                         "tiempo_mediana_ms_50":v["tiempo_mediana_ms_50"]}
                      for k,v in rutas.items()},indent=2))
    return datos


if __name__ == "__main__":
    ejecutar()
```